



Health Bite

Personalized Calorie & Restaurant Recommendation App

Project Title:	Health Bite – Personalized Calorie & Restaurant Recommendation App
Student 1:	Murad Ali Khan
Student 2:	Ahmed Zahid
Section:	EF23BSSE0219
Semester:	Advance Web Programming
Submission Date:	17th June 2026

React Native

Expo

OpenStreetMap

TheMealDB API

TypeScript

1. Project Overview

Health Bite is a cross-platform mobile application built with React Native (Expo) that helps users make informed food choices by calculating their daily calorie requirements and recommending nearby restaurants that match those targets. The app follows a simple, guided flow: users create an account, enter their physical metrics and fitness goal, receive a personalised TDEE (Total Daily Energy Expenditure) calculation, and are shown real local restaurants whose meals align with their calorie budget.

The application targets university students and health-conscious individuals who want practical, location-aware meal guidance without requiring a complex nutrition background. It bridges the gap between health science (Mifflin-St Jeor equation, BMI, TDEE) and everyday dining decisions.

Application Type:

Mobile Application (iOS & Android via Expo Go)
Target Users: University students, fitness beginners, health-conscious individuals

2. Problem Statement

Modern individuals, especially students with busy schedules, struggle to align their daily eating habits with their health and fitness goals. Existing calorie-counting apps require extensive manual food logging, and restaurant finder apps show no nutritional context. There is no unified solution that:

- Calculates a personalised calorie target based on the user's body metrics and goal (lose / maintain / gain)
- Shows a BMI score with visual feedback and actionable health advice
- Recommends real, nearby restaurants filtered to the user's calorie budget
- Allows restaurant owners to register and reach calorie-conscious customers

This gap results in uninformed food choices, wasted effort, and poor adherence to health goals.

3. Project Objectives

- Implement secure user registration and login with persistent session management
- Collect key physical metrics (age, weight, height, gender, activity level, fitness goal)
- Calculate TDEE using the Mifflin-St Jeor equation and adjust based on the selected goal
- Compute and display BMI with a colour-coded visual scale and personalised health tip
- Fetch real nearby restaurants using the free Overpass (OpenStreetMap) API
- Fall back to TheMealDB category-based meal suggestions when location is unavailable
- Allow restaurant owners to register their business and appear in user recommendations
- Provide a restaurant detail screen with website link and phone-call functionality
- Deliver a polished, dark-themed UI inspired by industry-leading apps (Uber, etc.)

4. Scope of the Project

Features Included

- User authentication (sign up, login, session persistence, logout)
- Profile setup: age, weight, height, gender, activity level, fitness goal
- TDEE calculation (Mifflin-St Jeor $\times$ activity multiplier)
- Goal-based calorie adjustment (" 500 / 0 / +300 kcal)
- BMI calculation with colour-coded scale bar and health category
- GPS-based restaurant discovery (3 km radius via Overpass API)
- TheMealDB fallback with TDEE-matched meal categories
- Restaurant detail screen: meal info, calorie estimate, star rating, website, phone
- Restaurant registration portal for business owners
- Registered restaurants displayed first with "Partner" badge
- Dark Uber-inspired UI with Inter font, green accent, animated landing screen

Features Excluded (Future Work)

- Backend database / cloud sync (currently uses on-device AsyncStorage only)
- Social features (share meals, follow friends)
- Full nutritional breakdown (macros, vitamins, allergens)
- In-app restaurant booking or ordering
- Push notifications for meal reminders

5. Functional Requirements

FR-1 Authentication

- Users can register with name, email, and password ("e6 characters)
- Users can log in with email and password
- Session is persisted across app restarts using AsyncStorage
- Users can log out, which clears all local session data

FR-2 Profile & Metrics

- Users enter age (10–100), weight (kg), height (cm), gender, activity level
- Users select a fitness goal: Lose Weight, Maintain, or Gain Muscle
- Profile is saved locally and reloaded on next launch

FR-3 Calorie & BMI Calculation

- TDEE calculated using Mifflin-St Jeor BMR $\times$ activity multiplier
- Adjusted target: Lose " 500 kcal · Maintain ± 0 · Gain +300 kcal
- BMI = $\text{weight(kg)} \div \text{height(m)}^2$; categorised as Underweight/Normal/Overweight/Obese

FR-4 Restaurant Recommendations

- App requests foreground location permission from the device
- Sends Overpass QL query; returns restaurants within 3 km
- Each card shows: restaurant name, meal name, estimated calories, star rating
- Cards are tappable and open a full detail screen

FR-5 Restaurant Registration

- Business owners fill a form: name, cuisine, address, city, phone, website, hours, price range,

description

- Data stored locally; registered restaurants appear first in results with Partner badge

6. Non-Functional Requirements

Usability

- Minimal onboarding — user reaches results in under 2 minutes
- Dark, high-contrast theme passes WCAG AA contrast ratios
- Haptic feedback on all interactive elements (Expo Haptics)
- Keyboard-aware layouts prevent input fields from being hidden

Performance

- React Query caches API responses for 10 minutes — no redundant network calls
- Expo Image lazy-loads with smooth 300 ms fade transitions
- FlatList virtualises the restaurant list for smooth 60 fps scrolling
- Animated landing screen uses React Native Animated API with native driver

Responsiveness

- Layouts use flex and SafeAreaInsets — adapts to any screen size and notch
- Runs on iOS 14+, Android 8+, and Expo Web (browser preview)

Security

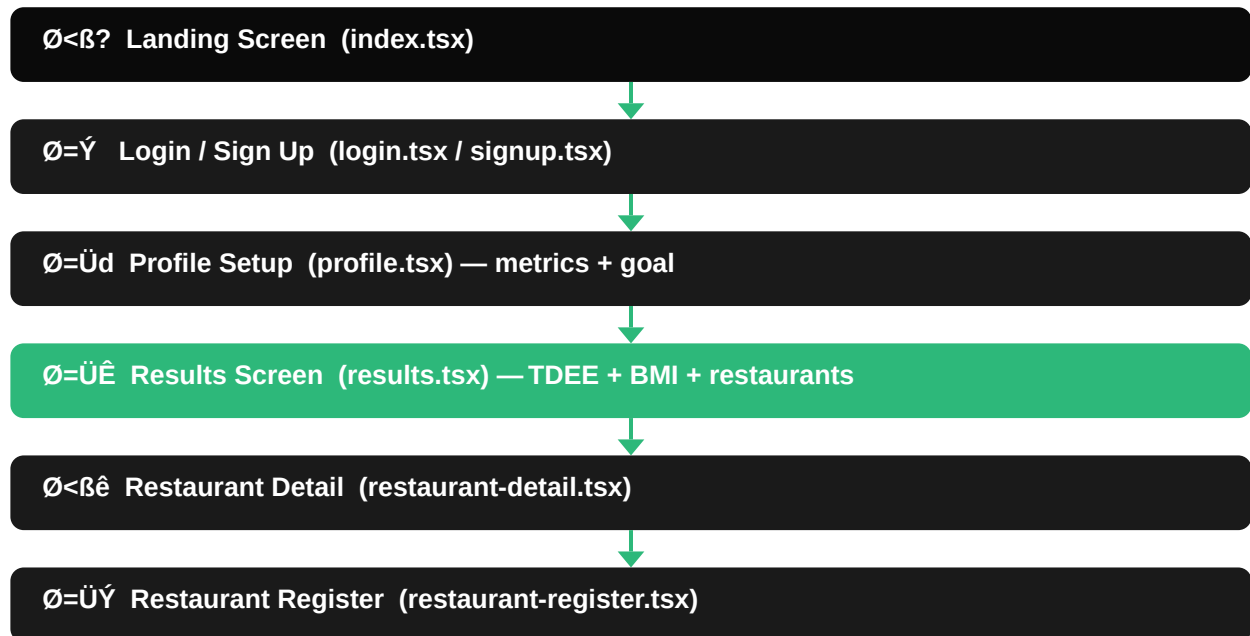
- No passwords are stored — email only persisted post-login (prototype scope)
- All API calls use HTTPS
- No sensitive data transmitted to third-party servers beyond location (Overpass)

7. Technology Stack

Framework:	React Native via Expo SDK 52
Navigation:	Expo Router v4 (file-based) Stack navigator
State Management:	React Context API + AsyncStorage (for react-native-async-storage)
Data Fetching:	React Query / react-query v5
Styling:	Expo Linear Gradient
Fonts:	Expo Font (font-size: 500 / 600 / 700)
Location:	Expo Location (foreground GPS)
Images:	Expo Image (high-performance lazy loading)
Package Manager:	Expo Packages (Expo Go app for device testing)
Primary API:	OpenMap - open StreetMap - free, no key required
Fallback API:	TheMapDB REST API V1 - free, no key required

8. Application Flow / Architecture

The application follows a linear onboarding flow with a persistent home screen. All screens live inside a single Expo Router Stack navigator under `app/(tabs)/`.



Data Flow

- App launch !' check AsyncStorage for existing login !' redirect to Results or Landing
- Profile saved !' TDEE + adjusted calories + BMI computed in-memory (utils/calories.ts)
- Results screen !' React Query fires API call !' Overpass (if location) or TheMealDB
- Registered restaurants loaded from AsyncStorage and merged at top of results list
- Tapping a card !' navigate to detail screen with all params passed via Expo Router

State Architecture

- AppContext — global auth state (isLoggedIn, email, userProfile) via React Context
- AsyncStorage — persistent storage for login flag, email, profile JSON, registered restaurants
- React Query — server-state cache with 10 min stale time for API responses

9. UI Screens / Components Description

1. Landing Screen

Purpose: First screen shown to new users. Introduces the app brand and guides users to sign up or log in.

- Animated floating food emoji icons (CSS keyframe-style loop)
- Green glowing logo mark (Ø<ß?) with gradient background
- "Eat Smart. Feel Great." hero headline
- Feature pills: Location-based · Calorie-smart · Rated meals
- Top-right Login / Sign Up buttons
- Bottom "Get Started !" white CTA button
- Auto-login check — redirects if session exists

2. Login Screen

Purpose: Allows existing users to authenticate. Validates email format and password length.

- Dark header with green logo and app name
- Email input with mail icon; highlights white border on focus
- Password input with lock icon (secureTextEntry)
- Inline validation error messages in red
- "Sign In" white primary button
- "Create New Account" outlined secondary button

3. Sign Up Screen

Purpose: New user registration. Collects name, email, password, and confirmation.

- Four input fields: Full Name, Email, Password, Confirm Password
- Each field has a Feather icon and focus highlight
- All fields validated before submission
- "Create Account !" primary button
- "Already have an account? Sign In" link at bottom

4. Profile Screen

Purpose: User enters physical metrics and selects fitness goal before calorie calculation.

- Age, Weight (kg), Height (cm) numeric inputs in dark cards
- Gender toggle: Male / Female (white active state)
- Goal picker: 3 cards — Ø=ÜÉ Lose Weight / &-p Maintain / Ø=Üª Gain Muscle
- Each goal card shows calorie delta (" 500 / ±0 / +300 kcal)
- Activity level: 5 radio options (Sedentary !' Very Active)
- "Calculate My Calories" white submit button

5. Results Screen

Purpose: Main home screen showing calorie target, BMI, and restaurant list.

- Top bar: settings (sliders icon) + logo + logout button
- Green TDEE card: adjusted calorie target, goal badge, per-meal estimate
- BMI card: score circle, colour-coded category, scale bar with marker, health tip
- Location permission banner (Enable !' requesting !' granted/denied states)
- "Own a restaurant?" registration CTA banner
-

Scrollable restaurant cards: photo, name, meal, calories, stars, Partner/Nearby badge

6. Restaurant Detail Screen

Purpose: Full information page for a selected restaurant. Opens when user taps a card.

- Full-width hero food image with gradient overlay
- Partner / Nearby badge overlaid on image
- Restaurant name, cuisine type, price range pill
- Star rating row with opening hours
- Calorie card: estimated kcal per meal + meal name
- "About" description card (for registered restaurants)
- Address, phone, website info row
- Large white "Visit Website" button with URL preview
- Green "Call" button (secondary)

7. Restaurant Registration Screen

Purpose: Allows restaurant owners to list their business in the app.

- Name input, cuisine chip selector (10 options), address, city fields
- Phone number and website URL inputs
- Price range selector: \$ / \$\$ / \$\$\$ / \$\$\$\$
- Opening hours input
- Multi-line description text area
- "Register Restaurant" submit button
- Success screen with green checkmark animation on completion

10. API Usage

API 1 — Overpass API (OpenStreetMap)

Base URL:

Root:

Query type:

The app builds an Overpass QL query that searches for all OpenStreetMap nodes tagged as amenity=restaurant within a 3 km radius of the device's GPS coordinates. Up to 15 results are returned. Each element includes the restaurant name (from OSM tags), cuisine type, and GPS coordinates (lat/lon). The calorie estimate per meal is calculated as $TDEE \div 3 \pm$ a small deterministic variation based on the OSM node ID.

- Endpoint example: `?data=[out:json];node["amenity"="restaurant"](around:3000,31.5,74.3);out body 15;`
- Response fields used: id, lat, lon, tags.name, tags.cuisine
- Fallback: if 0 results returned, TheMealDB is called instead

API 2 — TheMealDB (v1 — free tier)

Base URL:

Root:

Query type:

TDEE is mapped to a food category (Vegan / Vegetarian / Seafood / Chicken / Pasta / Beef) based on calorie thresholds. The API returns meals in that category, each with a name and thumbnail image URL. Calorie values are estimated deterministically from the meal ID within the category's known calorie range.

- Category mapping: `<1600 !' Vegan, <1900 !' Vegetarian, <2200 !' Seafood, <2500 !' Chicken, <2800 !' Pasta, 2800+ !' Beef`
- Response fields used: idMeal, strMeal, strMealThumb
- Up to 12 meals shown per query

Local Storage — AsyncStorage

No backend database is used in this prototype. All user data is stored on-device using `@react-native-async-storage/async-storage`:

- `isLoggedIn`, `userEmail` — authentication state
- `userProfile` — JSON object with all physical metrics and goal
- `registeredRestaurants` — JSON array of restaurant owner submissions

11. Conclusion

Health Bite successfully demonstrates a full mobile application lifecycle — from UI/UX design to data fetching and local persistence. By combining the Mifflin-St Jeor calorie equation with real-world restaurant data from OpenStreetMap, the app delivers genuinely personalised and actionable meal recommendations without requiring any paid APIs or backend infrastructure.

The project showcases practical skills in React Native, TypeScript, Expo Router navigation, REST API integration, React Query state management, and modern mobile UI design. The addition of goal-based calorie adjustment, BMI visual feedback, and a restaurant registration portal demonstrates the ability to extend a functional MVP with meaningful features.

Expected outcomes include a deeper understanding of cross-platform mobile development, API-driven UI design, and user experience principles. Future work would introduce a cloud backend (Supabase / Firebase), real menu data, macro tracking, and push notification reminders to make the app fully production-ready.

Submitted by